

M16 / LEARNER BOOK

DataOps, quality, security and observability

Standalone Zero-to-Job-Ready Data Engineer 2026

14 controlled lessons

A complete learning route

Start with the lesson video. Read the worked explanation beside it, pause to reproduce the example, then solve the supplied task. The books contain your study notes; keep your own work and results as evidence.

Use the protected answers after saving your first attempt. A changed retry shows what you can now do independently. There is no forced timer during learning.

One course. M00-M19. 264 lessons and 50 project phases.

Contents

Tap a topic or use the PDF bookmarks. These numbers are the actual pages in this edition.

START HERE | How to use this module 3

DE16-01 | Production mindset 4

DE16-02 | Testing strategy for data engineering 7

DE16-03 | Automated quality gates 10

DE16-04 | Continuous Integration fundamentals 13

DE16-05 | GitHub Actions for data projects 16

DE16-06 | Dev, test and prod environments 19

DE16-07 | Deployment and rollback 22

DE16-08 | Secrets and credential safety 25

DE16-09 | RBAC and least privilege 28

DE16-10 | PII, encryption and masking 31

DE16-11 | Metadata, lineage and governance 34

DE16-12 | Observability for data pipelines 37

DE16-13 | Incidents, runbooks and root cause 40

DE16-14 | Terraform and IaC foundations 43

START HERE

How to use this module

Open the matching practice folder from the course root: 15_M16/practice. Work on a copy. Commands printed in a lesson use that module as the working folder unless the lesson names a different location.

Study in lesson order. After the last lesson, use Quick Revision, take the Module Revision Test, then open its separate Protected Answers. Gates are additional assessments at the boundaries shown in the course guide.

Study rule: Use the learner book as the canonical route. Real videos are visible inside each lesson because this course is video-first; official/current documentation controls version-sensitive behavior. If a video has no independently verified bounded chapter, it stays available as an optional visual rather than being deleted.

Module outcome. You will learn to operate data systems as controlled production services: layered testing, quality gates, CI, GitHub Actions, environment promotion, rollback, secrets/OIDC, RBAC, PII controls, lineage, observability, incident response and Terraform/IaC foundations.

Runtime/tool note. This module is intentionally tool-boundary aware. GitHub Actions, Azure/Fabric, Great Expectations, OpenTelemetry and Terraform interfaces evolve. The concepts and supplied local simulators are stable learning evidence; current official docs are the authority for live commands, permissions and product UI.

Practice: 15_M16/practice/ Quick revision:

12_REVISION_AND_TESTS/M16/M16_Quick_Revision_Book.pdf Protected answers (open only after saved attempt): 12_REVISION_AND_TESTS/M16/PROTECTED_OPEN_AFTER_ATTEMPT/M16_Module_Revision_Test_Answers.pdf

DE16-01

Production mindset

Watch first

[1. HashiCorp - Introduction to Terraform](#)

0:30-3:26 IaC/Terraform; 3:26-10:13 Day 0/1/2; 21:19-22:23 providers

[2. Monte Carlo - Data Observability](#)

Optional visual - no verified bounded range

Pause to connect the demonstration to the example below. Use the printed stop time when a bounded range is given; longer recordings remain available for reference.

Lesson 1 of 14 | Primary practice: 15_M16/practice/dataops_patterns/01_production_contract.py

Why this matters

A pipeline that works once is a prototype. Production engineering begins when the same result must stay correct through retries, source changes, expired credentials, deployments, and downstream expectations. Your job is no longer just to make code run; it is to define the promise, collect evidence that the promise is being met, and recover safely when it is not.

Picture it in your head

Picture a restaurant kitchen serving 300 customers. Good cooking is necessary, but production also needs order tickets, food-safety checks, handoffs, inventory, ownership, and a recovery plan. A data pipeline is similar: transformation code is only one station in a larger operating system.

Words before we use them

New word	Meaning before we use it
production contract	A written statement of correctness, freshness, security and recovery promises.
SLO	A measurable reliability target such as freshness or success rate.
blast radius	How much data or business behavior one failure can affect.
runbook	A tested operational procedure for a known failure class.
recoverability	The ability to restore a correct state without inventing or losing accepted data.

Useful video / source route

[VIDEO: HashiCorp - Introduction to Terraform - Open video](#)

Watch: 0:30-3:26 IaC/Terraform; 3:26-10:13 Day 0/1/2; 21:19-22:23 providers

[VIDEO: Monte Carlo - Data Observability - Open video](#)

Watch: Optional visual - no verified bounded range

[OFFICIAL/CURRENT: oteI](#)

Use rule: Watch/read for the mental model or current interface, then return to this learner book and supplied practice. External media never replaces the independent task or validation evidence.

Follow with me once

Take a nightly orders pipeline. Write five promises before choosing tools: every accepted order appears once in curated data; paid revenue reconciles to source; data is ready by 06:00; secrets never appear in code/logs; and a bad release can be rolled back without deleting accepted source evidence. Map each promise to one independent proof. A green scheduler icon is not one of those proofs.

What you should see / why it worked

You should finish with a small production contract where another engineer can tell what healthy means, where to look for evidence, and which failure should stop publication. The reason this works is that the control is explicit, measurable and separated from hidden assumptions. A trustworthy result is not merely an output; it includes the boundary that was checked and evidence that can be reviewed later.

Break it: a realistic failure

At 05:40 the orchestration UI says success, but the gold table contains only 12% of the normal rows because the source API changed pagination. Compute succeeded; the business contract failed. A row-count/reconciliation gate must stop publication and preserve the previous good state.

Reason through the incident

Work in this order: 1) preserve the failing evidence; 2) identify which contract or boundary failed; 3) bound blast radius before changing state; 4) choose the smallest reversible repair; 5) rerun an independent validation; 6) record what would prevent recurrence. If you skip evidence and jump straight to a fix, you may restore the dashboard while losing the ability to explain why it failed.

Try it yourself

Create a production contract for a daily customer-status pipeline. Include correctness, freshness, access, deployment and recovery promises. Name one owner and one evidence source for each promise.

Fresh changed retry

Change the case to a streaming payments feed. Decide which promises need tighter latency, what replay/deduplication evidence is required, and what can remain eventually consistent.

Validation and recovery ladder

- PASS only when the required output and the named control both agree; a green command alone is insufficient.
- If the result is PARTIAL, first inspect inputs/configuration/permissions, then reproduce the smallest failing case.
- Never repair by deleting evidence or by granting broad privilege just to make an error disappear.
- After repair, change the fixture or scenario and prove the same principle still holds under a fresh case.

Revision in your own words

Core idea: Production mindset is about making production behavior explicit, reviewable and recoverable. The learner should be able to state the contract, show evidence, describe a failure boundary and perform a changed retry without copying the demonstration.

Teach it back

Explain why 'the DAG is green' is not enough to call a data product healthy. Give two examples of successful execution with wrong business output.

Evidence to keep

Save: the genuine first attempt, any error/failure output, the corrected result, one validation proof, and a short explanation of why the control protects production. Do not overwrite the first failed evidence with the repaired run.

DE16-02

Testing strategy for data engineering

Watch first

[1. TechWorld with Nana - GitHub Actions concepts and CI/CD with Docker](#)

Optional visual - no verified bounded range. English demonstration. Focus on workflow events, jobs, steps and seeing a failed check. Apply the course data-quality checks inside that workflow.

[2. Great Expectations tutorial](#)

Optional visual - no verified bounded range

[3. GitHub Actions desde cero - Spanish-language supplementary tutorial](#)

Optional Spanish-language visual - no verified bounded range. An English GitHub Actions demonstration is listed first.

Pause to connect the demonstration to the example below. Use the printed stop time when a bounded range is given; longer recordings remain available for reference.

Lesson 2 of 14 | Primary practice: 15_M16/practice/dataops_patterns/02_test_strategy.py

Why this matters

Data systems fail at many layers: a pure function can be wrong, a schema can drift, a database boundary can misbehave, a join can change grain, or totals can stop reconciling. One enormous end-to-end test is slow and hard to diagnose; only unit tests are too narrow. A useful strategy layers evidence by defect class.

Picture it in your head

Think of airport security. Identity, luggage, boarding eligibility and aircraft safety are separate checkpoints. No single checkpoint proves everything. Data testing needs the same defense-in-depth idea.

Words before we use them

New word	Meaning before we use it
unit test	A fast check of small deterministic logic.
integration test	A check across a real component boundary.
contract test	A check that an interface still has expected schema or semantics.
reconciliation	An independent comparison of counts, keys or business measures across stages.
fixture	Small controlled input used to make a test reproducible.

Useful video / source route

[VIDEO: Great Expectations tutorial - Open video](#)

Watch: Optional visual - no verified bounded range

[VIDEO: midudev - GitHub Actions tutorial - Open video](#)

Watch: Published chapters include 08:33 getting started, 19:00 deployment pipeline, 58:32 tokens/secrets, 01:28:40 parallel jobs

[OFFICIAL/CURRENT: gx](#)

[OFFICIAL/CURRENT: gx_validate](#)

Use rule: Watch/read for the mental model or current interface, then return to this learner book and supplied practice. External media never replaces the independent task or validation evidence.

Follow with me once

For order totals, first test the pure aggregation function with three rows. Then check required columns/types and non-null business keys. Next write the result to a disposable database. Finally reconcile source quantity*price against the curated sum. Each layer should tell you what class of defect it can catch and what it cannot prove.

What you should see / why it worked

A test matrix that names the test layer, fixture, command, expected result, runtime/cost, and defect class. Fast tests belong on every pull request; expensive environment checks belong at stronger release boundaries. The reason this works is that the control is explicit, measurable and separated from hidden assumptions. A trustworthy result is not merely an output; it includes the boundary that was checked and evidence that can be reviewed later.

Break it: a realistic failure

A refactor passes 65 unit tests but doubles daily revenue because a join changed grain and created fan-out. The missing proof was not another helper-function test; it was a grain/reconciliation assertion at the data boundary.

Reason through the incident

Work in this order: 1) preserve the failing evidence; 2) identify which contract or boundary failed; 3) bound blast radius before changing state; 4) choose the smallest reversible repair; 5) rerun an independent validation; 6) record what would prevent recurrence. If you skip evidence and jump straight to a fix, you may restore the dashboard while

losing the ability to explain why it failed.

Try it yourself

Build an eight-test matrix for ingestion -> staging -> curated. Use at least four layers and mark which tests run on every pull request versus before production promotion.

Fresh changed retry

The source adds nullable `coupon_code`. Decide what should continue to pass, what would fail if the field became required, and how backwards compatibility is demonstrated.

Validation and recovery ladder

- PASS only when the required output and the named control both agree; a green command alone is insufficient.
- If the result is PARTIAL, first inspect inputs/configuration/permissions, then reproduce the smallest failing case.
- Never repair by deleting evidence or by granting broad privilege just to make an error disappear.
- After repair, change the fixture or scenario and prove the same principle still holds under a fresh case.

Revision in your own words

Core idea: Testing strategy for data engineering is about making production behavior explicit, reviewable and recoverable. The learner should be able to state the contract, show evidence, describe a failure boundary and perform a changed retry without copying the demonstration.

Teach it back

Using one orders example, distinguish a unit test, a data-quality expectation, an integration test and a reconciliation check.

Evidence to keep

Save: the genuine first attempt, any error/failure output, the corrected result, one validation proof, and a short explanation of why the control protects production. Do not overwrite the first failed evidence with the repaired run.

DE16-03

Automated quality gates

Watch first

[1. TechWorld with Nana - GitHub Actions concepts and CI/CD with Docker](#)

Optional visual - no verified bounded range. English demonstration. Focus on workflow events, jobs, steps and seeing a failed check. Apply the course data-quality checks inside that workflow.

[2. Great Expectations tutorial](#)

Optional visual - no verified bounded range

[3. GitHub Actions desde cero - Spanish-language supplementary tutorial](#)

Optional Spanish-language visual - no verified bounded range. An English GitHub Actions demonstration is listed first.

Pause to connect the demonstration to the example below. Use the printed stop time when a bounded range is given; longer recordings remain available for reference.

Lesson 3 of 14 | Primary practice: 15_M16/practice/dataops_patterns/03_quality_gate.py

Why this matters

A validation report becomes operationally useful only when its result controls a decision. A quality gate turns measurements into pass, warn, quarantine or fail-closed behavior so technically processed but unfit data cannot quietly reach consumers.

Picture it in your head

A railway signal does not repair the track; it prevents a train from entering an unsafe block. A data quality gate does the same at a publish or deploy boundary.

Words before we use them

New word	Meaning before we use it
quality gate	A decision point that blocks or routes work based on explicit validation rules.
hard failure	A violation that stops publication or deployment.
warning	A recorded anomaly that does not block the flow.
quarantine	A safe retained location/state for rejected evidence.
threshold	A measurable boundary such as null-rate $\leq 0.5\%$.

Useful video / source route

[VIDEO: Great Expectations tutorial - Open video](#)

Watch: Optional visual - no verified bounded range

[VIDEO: midudev - GitHub Actions tutorial - Open video](#)

Watch: Published chapters include 08:33 getting started, 19:00 deployment pipeline, 58:32 tokens/secrets, 01:28:40 parallel jobs

[OFFICIAL/CURRENT: gx](#)

[OFFICIAL/CURRENT: gx_validate](#)

Use rule: Watch/read for the mental model or current interface, then return to this learner book and supplied practice. External media never replaces the independent task or validation evidence.

Follow with me once

For a customer batch define four rules: non-empty; customer_id unique/non-null; country in allowed set; row count at least 80% of rolling baseline. Run a good fixture and then a duplicate-ID fixture. The gate must emit machine-readable evidence, refuse publication on the hard failure, and retain the rejected batch for diagnosis.

What you should see / why it worked

A result containing rule name, measured value, threshold, severity and final decision. On a hard failure the committed accepted checkpoint must remain unchanged. The reason this works is that the control is explicit, measurable and separated from hidden assumptions. A trustworthy result is not merely an output; it includes the boundary that was checked and evidence that can be reviewed later.

Break it: a realistic failure

A source sends half the normal rows after an upstream timeout. Schema and uniqueness pass, but volume fails. If you advance the checkpoint before validation, replay becomes harder and bad progress is now accepted state. Correct behavior is quarantine + alert + no publish + no checkpoint advance.

Reason through the incident

Work in this order: 1) preserve the failing evidence; 2) identify which contract or boundary failed; 3) bound blast radius before changing state; 4) choose the smallest reversible repair; 5) rerun an independent validation; 6) record

what would prevent recurrence. If you skip evidence and jump straight to a fix, you may restore the dashboard while losing the ability to explain why it failed.

Try it yourself

Add a paid-revenue reconciliation rule and a freshness rule. Make one warning-only and one blocking, then justify the business consequence behind each severity.

Fresh changed retry

Change the volume baseline by 20% and show why the same incoming batch changes decision only because the contractual threshold changed. Record the rule version.

Validation and recovery ladder

- PASS only when the required output and the named control both agree; a green command alone is insufficient.
- If the result is PARTIAL, first inspect inputs/configuration/permissions, then reproduce the smallest failing case.
- Never repair by deleting evidence or by granting broad privilege just to make an error disappear.
- After repair, change the fixture or scenario and prove the same principle still holds under a fresh case.

Revision in your own words

Core idea: Automated quality gates is about making production behavior explicit, reviewable and recoverable. The learner should be able to state the contract, show evidence, describe a failure boundary and perform a changed retry without copying the demonstration.

Teach it back

Why is a validation report alone not a quality gate? Name the additional control that makes it a gate.

Evidence to keep

Save: the genuine first attempt, any error/failure output, the corrected result, one validation proof, and a short explanation of why the control protects production. Do not overwrite the first failed evidence with the repaired run.

DE16-04

Continuous Integration fundamentals

Watch first

[1. TechWorld with Nana - GitHub Actions concepts and CI/CD with Docker](#)

Optional visual - no verified bounded range. English demonstration. Focus on workflow events, jobs, steps and seeing a failed check. Apply the course data-quality checks inside that workflow.

[2. GitHub - Beginner 2026 compilation](#)

GitHub Actions 07:54-15:45; GitHub security 15:45-20:28 (publisher chapters)

[3. GitHub Actions desde cero - Spanish-language supplementary tutorial](#)

Optional Spanish-language visual - no verified bounded range. An English GitHub Actions demonstration is listed first.

Pause to connect the demonstration to the example below. Use the printed stop time when a bounded range is given; longer recordings remain available for reference.

Lesson 4 of 14 | Primary practice: 15_M16/practice/dataops_patterns/04_ci_simulator.py

Why this matters

Continuous Integration (CI) means small changes are integrated frequently and checked automatically before they damage the shared codebase. For data projects, CI can compile Python, run deterministic tests, validate SQL/configuration, check fixture-based contracts and scan for obvious secret leakage without needing production access.

Picture it in your head

Imagine a bridge inspected after each small repair rather than after six months of unreviewed changes. CI does not guarantee safe production, but it finds cheap defects while the change is still small and attributable.

Words before we use them

New word	Meaning before we use it
workflow	An automated sequence triggered by repository events.
runner	The machine that executes CI jobs.
job	A set of steps executed in one runner context.
artifact	A file produced by the workflow, such as a test report.
pull-request check	Automated evidence attached to a proposed change before merge.

Useful video / source route

[VIDEO: GitHub - Beginner 2026 compilation - Open video](#)

Watch: GitHub Actions 07:54-15:45; GitHub security 15:45-20:28 (publisher chapters)

[VIDEO: midudev - GitHub Actions tutorial - Open video](#)

Watch: Published chapters include 08:33 getting started, 19:00 deployment pipeline, 58:32 tokens/secrets, 01:28:40 parallel jobs

[OFFICIAL/CURRENT: gha_secure](#)

Use rule: Watch/read for the mental model or current interface, then return to this learner book and supplied practice. External media never replaces the independent task or validation evidence.

Follow with me once

Declare a minimal flow: checkout -> set up Python -> install locked dependencies -> compile -> unit tests -> fixture quality checks -> secret-pattern scan. Deliberately fail unit tests. The simulator should prove that later release steps are unreachable, then identify the failed stage in the evidence.

What you should see / why it worked

Every required check either passes or stops the workflow with a non-zero decision. A reviewer can see which gate failed without rerunning your laptop setup. The reason this works is that the control is explicit, measurable and separated from hidden assumptions. A trustworthy result is not merely an output; it includes the boundary that was checked and evidence that can be reviewed later.

Break it: a realistic failure

Common mistakes are running CI only after merge, hiding errors with continue-on-error, using floating dependencies, or putting powerful production credentials into a test workflow that never needed them. Workflow files themselves are executable code and must be reviewed as a trust boundary.

Reason through the incident

Work in this order: 1) preserve the failing evidence; 2) identify which contract or boundary failed; 3) bound blast radius before changing state; 4) choose the smallest reversible repair; 5) rerun an independent validation; 6) record what would prevent recurrence. If you skip evidence and jump straight to a fix, you may restore the dashboard while

losing the ability to explain why it failed.

Try it yourself

Write a CI matrix for the M16 practice repository: trigger, job, command, expected artifact and failure effect. Include at least one check requiring no network.

Fresh changed retry

First make the unit-test stage fail and prove deploy is unreachable. Then fix it and introduce a secret-pattern failure so a different gate blocks the path.

Validation and recovery ladder

- PASS only when the required output and the named control both agree; a green command alone is insufficient.
- If the result is PARTIAL, first inspect inputs/configuration/permissions, then reproduce the smallest failing case.
- Never repair by deleting evidence or by granting broad privilege just to make an error disappear.
- After repair, change the fixture or scenario and prove the same principle still holds under a fresh case.

Revision in your own words

Core idea: Continuous Integration fundamentals is about making production behavior explicit, reviewable and recoverable. The learner should be able to state the contract, show evidence, describe a failure boundary and perform a changed retry without copying the demonstration.

Teach it back

Explain CI to someone who believes it means 'automatically deploy every commit.' What is missing from that definition?

Evidence to keep

Save: the genuine first attempt, any error/failure output, the corrected result, one validation proof, and a short explanation of why the control protects production. Do not overwrite the first failed evidence with the repaired run.

DE16-05

GitHub Actions for data projects

Watch first

[1. TechWorld with Nana - GitHub Actions concepts and CI/CD with Docker](#)

Optional visual - no verified bounded range. English demonstration. Focus on workflow events, jobs, steps and seeing a failed check. Apply the course data-quality checks inside that workflow.

[2. GitHub - Beginner 2026 compilation](#)

GitHub Actions 07:54-15:45; GitHub security 15:45-20:28 (publisher chapters)

[3. GitHub Actions desde cero - Spanish-language supplementary tutorial](#)

Optional Spanish-language visual - no verified bounded range. An English GitHub Actions demonstration is listed first.

Pause to connect the demonstration to the example below. Use the printed stop time when a bounded range is given; longer recordings remain available for reference.

Lesson 5 of 14 | Primary practice: 15_M16/practice/dataops_patterns/05_github_actions_ci.yml

Why this matters

GitHub Actions is one practical CI/CD engine for repository events. The important skill is not memorizing YAML; it is understanding event -> job -> runner -> step -> permissions -> evidence so you can reason about what executes and what authority it has.

Picture it in your head

Think of a factory work order. The event opens the work order, the job is a station, the runner is the workstation, steps are operations, and permissions decide which locked cabinets that station may open.

Words before we use them

New word	Meaning before we use it
event	Repository activity that can trigger a workflow.
uses	Runs a reusable action.
run	Executes shell commands on the runner.
GITHUB_TOKEN	Short-lived token scoped to the workflow/repository.
OIDC	Federated identity mechanism that can avoid storing long-lived cloud credentials.

Useful video / source route

[VIDEO: GitHub - Beginner 2026 compilation - Open video](#)

Watch: GitHub Actions 07:54-15:45; GitHub security 15:45-20:28 (publisher chapters)

[VIDEO: midudev - GitHub Actions tutorial - Open video](#)

Watch: Published chapters include 08:33 getting started, 19:00 deployment pipeline, 58:32 tokens/secrets, 01:28:40 parallel jobs

[OFFICIAL/CURRENT: gha_secure](#)

[OFFICIAL/CURRENT: gha_oidc](#)

[OFFICIAL/CURRENT: gha_attest](#)

Use rule: Watch/read for the mental model or current interface, then return to this learner book and supplied practice. External media never replaces the independent task or validation evidence.

Follow with me once

Read the supplied workflow. It runs on pull_request and manual workflow_dispatch, grants contents: read, checks out code, sets up Python, compiles the practice folder and runs the quality gate. Notice that test CI needs no production secret. Then inspect where permissions are declared and why adding write-all would increase blast radius.

```
name: m16-ci
on:
  pull_request:
  workflow_dispatch:
permissions:
  contents: read
jobs:
  validate:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v6
      - uses: actions/setup-python@v5
        with:
          python-version: '3.13'
      - run: python -m compileall 15_M16/practice/dataops_patterns
      - run: python 15_M16/practice/dataops_patterns/03_quality_gate.py
```

What you should see / why it worked

You can identify trigger, job, runner image, reusable actions, shell commands, permissions and fail-closed checks. You can also explain which steps could run safely on an untrusted pull request. The reason this works is that the control is explicit, measurable and separated from hidden assumptions. A trustworthy result is not merely an output; it includes the boundary that was checked and evidence that can be reviewed later.

Break it: a realistic failure

A workflow uses a long-lived cloud key stored as a secret even though the provider supports OIDC. The key is copied into a debug command and appears in logs. The correct design is to remove unnecessary privilege, rotate/revoke leaked credentials, and prefer short-lived federated identity with explicit trust conditions.

Reason through the incident

Work in this order: 1) preserve the failing evidence; 2) identify which contract or boundary failed; 3) bound blast radius before changing state; 4) choose the smallest reversible repair; 5) rerun an independent validation; 6) record what would prevent recurrence. If you skip evidence and jump straight to a fix, you may restore the dashboard while losing the ability to explain why it failed.

Try it yourself

Create a branch-safe workflow that compiles code and runs two deterministic checks. Keep permissions read-only. Describe what additional controls would be required before adding deployment.

Fresh changed retry

Add a second job that depends on validate and produces a harmless text artifact. Break validate and prove the artifact job does not run; then repair it.

Validation and recovery ladder

- PASS only when the required output and the named control both agree; a green command alone is insufficient.
- If the result is PARTIAL, first inspect inputs/configuration/permissions, then reproduce the smallest failing case.
- Never repair by deleting evidence or by granting broad privilege just to make an error disappear.
- After repair, change the fixture or scenario and prove the same principle still holds under a fresh case.

Revision in your own words

Core idea: GitHub Actions for data projects is about making production behavior explicit, reviewable and recoverable. The learner should be able to state the contract, show evidence, describe a failure boundary and perform a changed retry without copying the demonstration.

Teach it back

What is the difference between `uses:` and `run:`? Why should workflow permissions be designed explicitly?

Evidence to keep

Save: the genuine first attempt, any error/failure output, the corrected result, one validation proof, and a short explanation of why the control protects production. Do not overwrite the first failed evidence with the repaired run.

DE16-06

Dev, test and prod environments

Watch first

[1. Fabric Git integration + Python CI/CD deployment](#)

Optional visual - no verified bounded range

[2. HashiCorp - Introduction to Terraform](#)

0:30-3:26 IaC/Terraform; 3:26-10:13 Day 0/1/2; 21:19-22:23 providers

Pause to connect the demonstration to the example below. Use the printed stop time when a bounded range is given; longer recordings remain available for reference.

Lesson 6 of 14 | Primary practice: 15_M16/practice/dataops_patterns/06_environment_binding.py

Why this matters

Environment separation limits blast radius and makes promotion reviewable. The goal is not three unrelated copies of a platform; it is the same tested artifact bound to different data, identities, endpoints and capacity according to environment policy.

Picture it in your head

Picture the same train model running first on a test track, then on a restricted certification track, then on the main network. The train design should not silently change between tracks; the track bindings and permissions do.

Words before we use them

New word	Meaning before we use it
environment	A controlled execution boundary such as dev, test or prod.
configuration	Values that vary without changing application logic.
promotion	Moving a tested version to a more trusted environment.
configuration drift	Unreviewed difference between intended and actual environment settings.
binding	Connection from an artifact to environment-specific resources.

Useful video / source route

[VIDEO: Fabric Git integration + Python CI/CD deployment - Open video](#)

Watch: Optional visual - no verified bounded range

[VIDEO: HashiCorp - Introduction to Terraform - Open video](#)

Watch: 0:30-3:26 IaC/Terraform; 3:26-10:13 Day 0/1/2; 21:19-22:23 providers

[OFFICIAL/CURRENT: fabric_cicd](#)

[OFFICIAL/CURRENT: fabric_git](#)

Use rule: Watch/read for the mental model or current interface, then return to this learner book and supplied practice. External media never replaces the independent task or validation evidence.

Follow with me once

Use one pipeline artifact and three configuration maps. Dev reads synthetic/local data, test reads a controlled integration source, prod reads the approved production source. The binding checker rejects a configuration where the test artifact points at the production customer table. This is environment safety expressed as code.

What you should see / why it worked

You can show the same artifact/version across environments while endpoints, identities and capacities change through explicit configuration. No secret value is baked into the artifact. The reason this works is that the control is explicit, measurable and separated from hidden assumptions. A trustworthy result is not merely an output; it includes the boundary that was checked and evidence that can be reviewed later.

Break it: a realistic failure

A test pipeline is copied manually from production and still references the prod sink. A harmless test run overwrites business data. The root cause is not 'someone clicked wrong'; it is missing environment binding controls and excessive test identity permissions.

Reason through the incident

Work in this order: 1) preserve the failing evidence; 2) identify which contract or boundary failed; 3) bound blast radius before changing state; 4) choose the smallest reversible repair; 5) rerun an independent validation; 6) record

what would prevent recurrence. If you skip evidence and jump straight to a fix, you may restore the dashboard while losing the ability to explain why it failed.

Try it yourself

Design dev/test/prod bindings for a bronze-to-gold pipeline. Include source, sink, identity, capacity and data-sensitivity differences.

Fresh changed retry

Swap one test binding to a production resource and prove your checker rejects it. Then add an approved read-only prod reference and explain why its risk differs.

Validation and recovery ladder

- PASS only when the required output and the named control both agree; a green command alone is insufficient.

- If the result is PARTIAL, first inspect inputs/configuration/permissions, then reproduce the smallest failing case.
- Never repair by deleting evidence or by granting broad privilege just to make an error disappear.
- After repair, change the fixture or scenario and prove the same principle still holds under a fresh case.

Revision in your own words

Core idea: Dev, test and prod environments is about making production behavior explicit, reviewable and recoverable. The learner should be able to state the contract, show evidence, describe a failure boundary and perform a changed retry without copying the demonstration.

Teach it back

Why should promotion prefer the same artifact with different bindings rather than three manually edited copies?

Evidence to keep

Save: the genuine first attempt, any error/failure output, the corrected result, one validation proof, and a short explanation of why the control protects production. Do not overwrite the first failed evidence with the repaired run.

DE16-07

Deployment and rollback

Watch first

[1. TechWorld with Nana - GitHub Actions concepts and CI/CD with Docker](#)

Optional visual - no verified bounded range. English demonstration. Focus on workflow events, jobs, steps and seeing a failed check. Apply the course data-quality checks inside that workflow.

[2. Fabric Git integration + Python CI/CD deployment](#)

Optional visual - no verified bounded range

[3. GitHub Actions desde cero - Spanish-language supplementary tutorial](#)

Optional Spanish-language visual - no verified bounded range. An English GitHub Actions demonstration is listed first.

Pause to connect the demonstration to the example below. Use the printed stop time when a bounded range is given; longer recordings remain available for reference.

Lesson 7 of 14 | Primary practice:

15_M16/practice/dataops_patterns/07_deploy_rollback_simulator.py

Why this matters

Deployment changes running behavior; rollback is the controlled path back when a release proves unsafe. Data engineering adds a complication: rolling code back does not automatically undo data that the bad code already wrote. Release design must separate code rollback, data repair and replay.

Picture it in your head

Imagine replacing a machine on an assembly line. You can reinstall the old machine, but defective products already produced do not disappear. They need identification, quarantine and reprocessing.

Words before we use them

New word	Meaning before we use it
deployment	Promotion of a version/configuration into an environment.
rollback	Returning code/configuration to a previously known-good version.
roll forward	Fixing by deploying a new corrected version.
migration	A controlled change to stored schema/state.
expand-contract	Compatibility pattern that introduces changes before removing old behavior.

Useful video / source route

[VIDEO: Fabric Git integration + Python CI/CD deployment - Open video](#)

Watch: Optional visual - no verified bounded range

[VIDEO: midudev - GitHub Actions tutorial - Open video](#)

Watch: Published chapters include 08:33 getting started, 19:00 deployment pipeline, 58:32 tokens/secrets, 01:28:40 parallel jobs

[OFFICIAL/CURRENT: fabric_cicd](#)

Use rule: Watch/read for the mental model or current interface, then return to this learner book and supplied practice. External media never replaces the independent task or validation evidence.

Follow with me once

Simulate release v2. Smoke tests pass, but the first production reconciliation fails. Mark v2 unhealthy, restore v1 code, identify rows written by v2, and decide whether they can be deterministically replayed. The evidence should distinguish code state from data state.

What you should see / why it worked

A deployment record naming version, environment, checks, decision, rollback target and affected data window. Rollback restores code safely without pretending data repair is automatic. The reason this works is that the control is explicit, measurable and separated from hidden assumptions. A trustworthy result is not merely an output; it includes the boundary that was checked and evidence that can be reviewed later.

Break it: a realistic failure

A destructive schema migration drops a column before all consumers are updated. Rolling application code back cannot recreate the deleted values. Safer changes often use expand-contract: add compatible structure, migrate/verify readers, then remove old structure in a later controlled release.

Reason through the incident

Work in this order: 1) preserve the failing evidence; 2) identify which contract or boundary failed; 3) bound blast radius before changing state; 4) choose the smallest reversible repair; 5) rerun an independent validation; 6) record what would prevent recurrence. If you skip evidence and jump straight to a fix, you may restore the dashboard while losing the ability to explain why it failed.

Try it yourself

Write a release checklist for a pipeline with a Delta table and one downstream dashboard. Include pre-deploy evidence, smoke/reconciliation checks and data-repair decision points.

Fresh changed retry

Change the case so v2 writes an idempotent derived table rather than destructive state. Explain why replay is now easier and what evidence still matters.

Validation and recovery ladder

- PASS only when the required output and the named control both agree; a green command alone is insufficient.
- If the result is PARTIAL, first inspect inputs/configuration/permissions, then reproduce the smallest failing case.
- Never repair by deleting evidence or by granting broad privilege just to make an error disappear.
- After repair, change the fixture or scenario and prove the same principle still holds under a fresh case.

Revision in your own words

Core idea: Deployment and rollback is about making production behavior explicit, reviewable and recoverable. The learner should be able to state the contract, show evidence, describe a failure boundary and perform a changed retry without copying the demonstration.

Teach it back

Why is code rollback not the same as data rollback? Give one example where the old code is restored but the system remains wrong.

Evidence to keep

Save: the genuine first attempt, any error/failure output, the corrected result, one validation proof, and a short explanation of why the control protects production. Do not overwrite the first failed evidence with the repaired run.

DE16-08

Secrets and credential safety

Watch first

[1. TechWorld with Nana - GitHub Actions concepts and CI/CD with Docker](#)

Optional visual - no verified bounded range. English demonstration. Focus on workflow events, jobs, steps and seeing a failed check. Apply the course data-quality checks inside that workflow.

[2. GitHub - Beginner 2026 compilation](#)

GitHub Actions 07:54-15:45; GitHub security 15:45-20:28 (publisher chapters)

[3. GitHub Actions desde cero - Spanish-language supplementary tutorial](#)

Optional Spanish-language visual - no verified bounded range. An English GitHub Actions demonstration is listed first.

Pause to connect the demonstration to the example below. Use the printed stop time when a bounded range is given; longer recordings remain available for reference.

Lesson 8 of 14 | Primary practice: 15_M16/practice/dataops_patterns/08_secret_hygiene.py

Why this matters

Credentials are authority. A leaked secret is not just untidy text; it can let an attacker or accidental process act as your system. Modern cloud workflows therefore prefer managed identities or short-lived federation such as OIDC over copied long-lived keys whenever the platform supports it.

Picture it in your head

A password written on a sticky note is dangerous because anyone who sees it inherits your authority. A managed identity is closer to a guarded badge issued to a known workload for a limited purpose and lifetime.

Words before we use them

New word	Meaning before we use it
secret	Sensitive authentication material such as token, key or password.
managed identity	Cloud identity assigned to a workload instead of embedding credentials.
OIDC federation	Trust exchange that can issue short-lived provider credentials based on workflow identity.
rotation	Replacing credential material and invalidating the old value.
trust condition	Rule restricting which identity claims are allowed to obtain credentials.

Useful video / source route

[VIDEO: GitHub - Beginner 2026 compilation - Open video](#)

Watch: GitHub Actions 07:54-15:45; GitHub security 15:45-20:28 (publisher chapters)

[VIDEO: midudev - GitHub Actions tutorial - Open video](#)

Watch: Published chapters include 08:33 getting started, 19:00 deployment pipeline, 58:32 tokens/secrets, 01:28:40 parallel jobs

[OFFICIAL/CURRENT: gha_oidc](#)

[OFFICIAL/CURRENT: gha_secrets](#)

[OFFICIAL/CURRENT: managed_identity](#)

Use rule: Watch/read for the mental model or current interface, then return to this learner book and supplied practice. External media never replaces the independent task or validation evidence.

Follow with me once

Run the hygiene scanner over sample configuration. It should flag obvious embedded tokens but not treat the scanner as complete protection. Then sketch a GitHub Actions -> cloud OIDC flow: workflow identity token, cloud trust policy, short-lived role/session, least-privilege access. No long-lived cloud key is committed to the repository.

What you should see / why it worked

You can explain where secret material may leak (code, git history, logs, notebooks, config exports) and what to do after exposure: revoke/rotate, investigate use, remove from future code/history where appropriate, and replace the design. The reason this works is that the control is explicit, measurable and separated from hidden assumptions. A trustworthy result is not merely an output; it includes the boundary that was checked and evidence that can be reviewed later.

Break it: a realistic failure

A key is accidentally committed and then 'fixed' by deleting the line in the next commit. The key still exists in history and may already have been copied. Treat it as compromised immediately; revoke/rotate first. Cleanup is secondary to removing authority.

Reason through the incident

Work in this order: 1) preserve the failing evidence; 2) identify which contract or boundary failed; 3) bound blast radius before changing state; 4) choose the smallest reversible repair; 5) rerun an independent validation; 6) record what would prevent recurrence. If you skip evidence and jump straight to a fix, you may restore the dashboard while losing the ability to explain why it failed.

Try it yourself

Take a hypothetical CI workflow currently using AZURE_CLIENT_SECRET. Redesign it around OIDC/managed identity if supported. List trust conditions and least-privilege permissions.

Fresh changed retry

Assume OIDC is temporarily unavailable for one legacy service. Design a contained secret fallback with rotation, restricted scope, masked logs and an explicit removal date.

Validation and recovery ladder

- PASS only when the required output and the named control both agree; a green command alone is insufficient.
- If the result is PARTIAL, first inspect inputs/configuration/permissions, then reproduce the smallest failing case.
- Never repair by deleting evidence or by granting broad privilege just to make an error disappear.
- After repair, change the fixture or scenario and prove the same principle still holds under a fresh case.

Revision in your own words

Core idea: Secrets and credential safety is about making production behavior explicit, reviewable and recoverable. The learner should be able to state the contract, show evidence, describe a failure boundary and perform a changed retry without copying the demonstration.

Teach it back

Why does deleting a leaked credential from the latest source file not make the credential safe again?

Evidence to keep

Save: the genuine first attempt, any error/failure output, the corrected result, one validation proof, and a short explanation of why the control protects production. Do not overwrite the first failed evidence with the repaired run.

DE16-09

RBAC and least privilege

Watch first

[1. Azure RBAC visual walkthrough](#)

00:26-04:18 RBAC core through best practices

[2. GitHub - Beginner 2026 compilation](#)

GitHub Actions 07:54-15:45; GitHub security 15:45-20:28 (publisher chapters)

Pause to connect the demonstration to the example below. Use the printed stop time when a bounded range is given; longer recordings remain available for reference.

Lesson 9 of 14 | Primary practice: 15_M16/practice/dataops_patterns/09_rbac_least_privilege.py

Why this matters

Role-based access control (RBAC) limits who can do what at which scope. Least privilege means granting only the actions and resource scope needed for the task, then reviewing access as responsibilities change. It reduces the blast radius of mistakes and stolen identities.

Picture it in your head

In a hotel, a guest key opens one room for a time; housekeeping keys open assigned rooms; a master key is rare and audited. Giving every employee the master key is convenient until the first mistake or loss.

Words before we use them

New word	Meaning before we use it
principal	User, group, service principal or managed identity receiving access.
role	Named set of allowed actions.
scope	Resource boundary where the assignment applies.
data plane	Operations on data/content.
control plane	Operations that configure/manage the resource itself.

Useful video / source route

[VIDEO: Azure RBAC visual walkthrough - Open video](#)

Watch: 00:26-04:18 RBAC core through best practices

[VIDEO: GitHub - Beginner 2026 compilation - Open video](#)

Watch: GitHub Actions 07:54-15:45; GitHub security 15:45-20:28 (publisher chapters)

[OFFICIAL/CURRENT: keyvault_rbac](#)

[OFFICIAL/CURRENT: managed_identity](#)

Use rule: Watch/read for the mental model or current interface, then return to this learner book and supplied practice. External media never replaces the independent task or validation evidence.

Follow with me once

Model three principals: ingestion identity writes raw storage, transformation identity reads raw/writes curated, analyst identity reads curated. The policy checker rejects Owner at subscription scope for all three. Then compare a management role with a data-access role: being able to configure a Key Vault is not automatically the same as being allowed to read every secret.

What you should see / why it worked

A role matrix where principal, action, scope and justification are explicit. Broad roles have documented exception reasoning rather than being default troubleshooting tools. The reason this works is that the control is explicit, measurable and separated from hidden assumptions. A trustworthy result is not merely an output; it includes the boundary that was checked and evidence that can be reviewed later.

Break it: a realistic failure

A pipeline receives 403 and the quickest fix is to grant Owner at subscription scope. The job works, but the identity can now change far more than necessary. Correct debugging identifies the exact denied operation and grants the narrow data/control permission at the smallest workable scope.

Reason through the incident

Work in this order: 1) preserve the failing evidence; 2) identify which contract or boundary failed; 3) bound blast radius before changing state; 4) choose the smallest reversible repair; 5) rerun an independent validation; 6) record what would prevent recurrence. If you skip evidence and jump straight to a fix, you may restore the dashboard while losing the ability to explain why it failed.

Try it yourself

Design roles for source ingestion, transformation, deployment and read-only analytics. State which role must never read PII raw data and why.

Fresh changed retry

Change the transformation task so it needs to rotate a secret. Decide whether to expand its existing role or delegate rotation to a separate automation identity.

Validation and recovery ladder

- PASS only when the required output and the named control both agree; a green command alone is insufficient.

- If the result is PARTIAL, first inspect inputs/configuration/permissions, then reproduce the smallest failing case.
- Never repair by deleting evidence or by granting broad privilege just to make an error disappear.
- After repair, change the fixture or scenario and prove the same principle still holds under a fresh case.

Revision in your own words

Core idea: RBAC and least privilege is about making production behavior explicit, reviewable and recoverable. The learner should be able to state the contract, show evidence, describe a failure boundary and perform a changed retry without copying the demonstration.

Teach it back

Explain principal + role + scope with one Azure example. Why can a management-plane role differ from data access?

Evidence to keep

Save: the genuine first attempt, any error/failure output, the corrected result, one validation proof, and a short explanation of why the control protects production. Do not overwrite the first failed evidence with the repaired run.

DE16-10

PII, encryption and masking

Watch first

[1. Microsoft Purview Learn Live](#)

03:44-15:47 permissions; 40:21-52:44 lineage; 52:44-1:05:22 classification

[2. Azure RBAC visual walkthrough](#)

00:26-04:18 RBAC core through best practices

Pause to connect the demonstration to the example below. Use the printed stop time when a bounded range is given; longer recordings remain available for reference.

Lesson 10 of 14 | Primary practice: 15_M16/practice/dataops_patterns/10_pii_masking.py

Why this matters

Personal data creates a different risk class. Protecting it requires knowing where it exists, minimizing unnecessary copies, encrypting storage and transport, controlling access, and ensuring logs/test fixtures/analytics do not expose raw identifiers without a business need.

Picture it in your head

A hospital does not post patient charts in the hallway merely because the building door is locked. Storage encryption is one layer; who can view which field and where copies appear is another.

Words before we use them

New word	Meaning before we use it
PII	Personally identifiable information that can identify or be linked to a person.
encryption at rest	Protection of stored bytes using cryptographic keys.
encryption in transit	Protection while data moves across networks.
masking	Showing a transformed/redacted representation instead of raw sensitive value.
minimization	Keeping only data needed for the defined purpose.

Useful video / source route

[VIDEO: Microsoft Purview Learn Live - Open video](#)

Watch: 03:44-15:47 permissions; 40:21-52:44 lineage; 52:44-1:05:22 classification

[VIDEO: Azure RBAC visual walkthrough - Open video](#)

Watch: 00:26-04:18 RBAC core through best practices

[OFFICIAL/CURRENT: storage_encrypt](#)

[OFFICIAL/CURRENT: purview_roles](#)

Use rule: Watch/read for the mental model or current interface, then return to this learner book and supplied practice. External media never replaces the independent task or validation evidence.

Follow with me once

Load a safe customer fixture containing synthetic email and phone values. The masking function emits a support-safe view while preserving a stable non-PII customer key. Compare four controls: source minimization, storage encryption, access control and output masking. None substitutes for the others.

What you should see / why it worked

You can classify fields, identify raw versus masked zones, state who needs unmasked access, and explain how logs/telemetry avoid recording PII values. The reason this works is that the control is explicit, measurable and separated from hidden assumptions. A trustworthy result is not merely an output; it includes the boundary that was checked and evidence that can be reviewed later.

Break it: a realistic failure

A pipeline correctly encrypts storage but prints full email addresses and authentication headers to logs during debugging. The logs are copied to a wider observability platform. Encryption at rest did not prevent the exposure because the sensitive values were intentionally emitted after decryption.

Reason through the incident

Work in this order: 1) preserve the failing evidence; 2) identify which contract or boundary failed; 3) bound blast radius before changing state; 4) choose the smallest reversible repair; 5) rerun an independent validation; 6) record

what would prevent recurrence. If you skip evidence and jump straight to a fix, you may restore the dashboard while losing the ability to explain why it failed.

Try it yourself

Classify ten fields in a customer/orders dataset as public, internal, confidential or sensitive PII. Design masked analyst outputs for the PII fields.

Fresh changed retry

Add a legitimate fraud-use case requiring a small group to see raw phone numbers. Define the narrow access, audit and retention controls rather than weakening every dataset.

Validation and recovery ladder

- PASS only when the required output and the named control both agree; a green command alone is insufficient.

- If the result is PARTIAL, first inspect inputs/configuration/permissions, then reproduce the smallest failing case.
- Never repair by deleting evidence or by granting broad privilege just to make an error disappear.
- After repair, change the fixture or scenario and prove the same principle still holds under a fresh case.

Revision in your own words

Core idea: PII, encryption and masking is about making production behavior explicit, reviewable and recoverable. The learner should be able to state the contract, show evidence, describe a failure boundary and perform a changed retry without copying the demonstration.

Teach it back

Why are encryption and masking different controls? Give one failure that encryption alone does not prevent.

Evidence to keep

Save: the genuine first attempt, any error/failure output, the corrected result, one validation proof, and a short explanation of why the control protects production. Do not overwrite the first failed evidence with the repaired run.

DE16-11

Metadata, lineage and governance

Watch first

[1. Microsoft Purview Learn Live](#)

03:44-15:47 permissions; 40:21-52:44 lineage; 52:44-1:05:22 classification

[2. Microsoft Purview lineage visual](#)

Optional visual - no verified bounded range

Pause to connect the demonstration to the example below. Use the printed stop time when a bounded range is given; longer recordings remain available for reference.

Lesson 11 of 14 | Primary practice: 15_M16/practice/dataops_patterns/11_lineage_graph.py

Why this matters

When a field is wrong, teams need to know where it came from, what transformed it, who owns it, what consumes it and whether it is sensitive. Metadata and lineage turn a collection of tables into an explainable data system and make change impact review possible.

Picture it in your head

Imagine a subway map. A station name alone is useful, but the lines show how stations connect, where transfers happen and what downstream routes are affected by a closure. Lineage is that connectivity map for data assets.

Words before we use them

New word	Meaning before we use it
metadata	Information describing a data asset, such as owner, schema, grain or sensitivity.
lineage	Recorded upstream/downstream relationships between assets and transformations.
governance	Policies, roles and processes for trustworthy and appropriate data use.
catalog	Searchable inventory of data assets and metadata.
impact analysis	Reasoning about what downstream assets may be affected by a change.

Useful video / source route

[VIDEO: Microsoft Purview Learn Live - Open video](#)

Watch: 03:44-15:47 permissions; 40:21-52:44 lineage; 52:44-1:05:22 classification

[VIDEO: Microsoft Purview lineage visual - Open video](#)

Watch: Optional visual - no verified bounded range

[OFFICIAL/CURRENT: purview_roles](#)

[OFFICIAL/CURRENT: purview_lineage](#)

Use rule: Watch/read for the mental model or current interface, then return to this learner book and supplied practice. External media never replaces the independent task or validation evidence.

Follow with me once

Build a graph: orders_api -> raw_orders -> silver_orders -> gold_daily_sales -> executive_dashboard. Attach owner, grain and sensitivity to each node. Then simulate removing discount_amount from silver. Traverse downstream edges and list assets needing review before the change is approved.

What you should see / why it worked

A lineage graph that supports both root-cause questions ('where did this come from?') and impact questions ('what will break if I change this?'). Known lineage gaps are stated instead of hidden. The reason this works is that the control is explicit, measurable and separated from hidden assumptions. A trustworthy result is not merely an output; it includes the boundary that was checked and evidence that can be reviewed later.

Break it: a realistic failure

A catalog lists every table but no owners, transformation relationships or sensitive classifications. Search works, but incident routing and impact analysis do not. Governance requires usable metadata and operating responsibility, not just inventory count.

Reason through the incident

Work in this order: 1) preserve the failing evidence; 2) identify which contract or boundary failed; 3) bound blast radius before changing state; 4) choose the smallest reversible repair; 5) rerun an independent validation; 6) record

what would prevent recurrence. If you skip evidence and jump straight to a fix, you may restore the dashboard while losing the ability to explain why it failed.

Try it yourself

Draw lineage for one pipeline you built earlier. Add owner, grain, freshness expectation and sensitivity for each major asset.

Fresh changed retry

Insert a second consumer and a shortcut/external table whose lineage is only partially captured. Mark the gap and document a manual control until automated lineage is available.

Validation and recovery ladder

- PASS only when the required output and the named control both agree; a green command alone is insufficient.
- If the result is PARTIAL, first inspect inputs/configuration/permissions, then reproduce the smallest failing case.
- Never repair by deleting evidence or by granting broad privilege just to make an error disappear.
- After repair, change the fixture or scenario and prove the same principle still holds under a fresh case.

Revision in your own words

Core idea: Metadata, lineage and governance is about making production behavior explicit, reviewable and recoverable. The learner should be able to state the contract, show evidence, describe a failure boundary and perform a changed retry without copying the demonstration.

Teach it back

What can lineage answer that a schema alone cannot? Give one upstream root-cause and one downstream impact question.

Evidence to keep

Save: the genuine first attempt, any error/failure output, the corrected result, one validation proof, and a short explanation of why the control protects production. Do not overwrite the first failed evidence with the repaired run.

DE16-12

Observability for data pipelines

Watch first

[1. Monte Carlo - Data Observability](#)

Optional visual - no verified bounded range

[2. Google Cloud Tech - OpenTelemetry](#)

Optional visual - no verified bounded range

Pause to connect the demonstration to the example below. Use the printed stop time when a bounded range is given; longer recordings remain available for reference.

Lesson 12 of 14 | Primary practice:

15_M16/practice/dataops_patterns/12_observability_metrics.py

Why this matters

Monitoring tells you that a metric crossed a threshold; observability helps you understand system state from evidence. Data pipelines need ordinary software signals plus data-specific signals such as freshness, volume, schema, reconciliation and distribution because compute can look healthy while the data is wrong.

Picture it in your head

A car dashboard can show engine temperature and fuel, but if you are transporting medicine you also need cargo temperature. Data systems likewise need infrastructure telemetry and data-product telemetry.

Words before we use them

New word	Meaning before we use it
metric	Numeric measurement tracked over time.
log	Timestamped event record.
trace	Context connecting work across components.
freshness	How current a dataset is relative to expectation.
data observability	Evidence about data health across freshness, volume, schema, quality and lineage/context.

Useful video / source route

[VIDEO: Monte Carlo - Data Observability - Open video](#)

Watch: Optional visual - no verified bounded range

[VIDEO: Google Cloud Tech - OpenTelemetry - Open video](#)

Watch: Optional visual - no verified bounded range

[OFFICIAL/CURRENT: otel](#)

Use rule: Watch/read for the mental model or current interface, then return to this learner book and supplied practice. External media never replaces the independent task or validation evidence.

Follow with me once

Generate a health snapshot containing run duration, error count, output rows, freshness minutes and reconciliation delta. Keep runtime normal but drop output rows by 80%. Software telemetry alone looks fine; data-volume/reconciliation telemetry reveals the real failure. Add an alert decision that names both measured value and expected boundary.

What you should see / why it worked

A compact health record with software and data signals, enough context to route an incident, and no raw PII as high-cardinality telemetry labels. The reason this works is that the control is explicit, measurable and separated from hidden assumptions. A trustworthy result is not merely an output; it includes the boundary that was checked and evidence that can be reviewed later.

Break it: a realistic failure

Teams emit `customer_email` as a metric label to debug individual failures. This leaks PII and creates huge cardinality/cost. Telemetry should use bounded technical identifiers and link to appropriately secured investigation data when necessary.

Reason through the incident

Work in this order: 1) preserve the failing evidence; 2) identify which contract or boundary failed; 3) bound blast radius before changing state; 4) choose the smallest reversible repair; 5) rerun an independent validation; 6) record what would prevent recurrence. If you skip evidence and jump straight to a fix, you may restore the dashboard while

losing the ability to explain why it failed.

Try it yourself

Define eight signals for a daily revenue pipeline: at least two software signals, four data-health signals, one dependency signal and one business reconciliation signal.

Fresh changed retry

Change the incident so row count is normal but freshness is six hours late. Decide which alert should fire and why a pure volume dashboard would miss it.

Validation and recovery ladder

- PASS only when the required output and the named control both agree; a green command alone is insufficient.

- If the result is PARTIAL, first inspect inputs/configuration/permissions, then reproduce the smallest failing case.
- Never repair by deleting evidence or by granting broad privilege just to make an error disappear.
- After repair, change the fixture or scenario and prove the same principle still holds under a fresh case.

Revision in your own words

Core idea: Observability for data pipelines is about making production behavior explicit, reviewable and recoverable. The learner should be able to state the contract, show evidence, describe a failure boundary and perform a changed retry without copying the demonstration.

Teach it back

Why can CPU/runtime/success metrics all be healthy while a data product is unhealthy? Name three data-specific signals.

Evidence to keep

Save: the genuine first attempt, any error/failure output, the corrected result, one validation proof, and a short explanation of why the control protects production. Do not overwrite the first failed evidence with the repaired run.

DE16-13

Incidents, runbooks and root cause

Watch first

[1. Google Cloud Tech - OpenTelemetry](#)

Optional visual - no verified bounded range

[2. Monte Carlo - Data Observability](#)

Optional visual - no verified bounded range

Pause to connect the demonstration to the example below. Use the printed stop time when a bounded range is given; longer recordings remain available for reference.

Lesson 13 of 14 | Primary practice: [15_M16/practice/dataops_patterns/13_incident_rca.py](#)

Why this matters

Production incidents are time-constrained reasoning problems. The goal is first to protect consumers and restore a correct service, then to understand causes and strengthen the system. Good incident handling separates facts from hypotheses and preserves evidence instead of improvising irreversible fixes.

Picture it in your head

Firefighters first contain the fire and protect people; the detailed origin investigation follows after the scene is safe. Data incidents need the same separation between containment/recovery and root-cause analysis.

Words before we use them

New word	Meaning before we use it
incident	Unplanned event that degrades correctness, availability, security or service expectations.
containment	Action limiting further harm before full cause is known.
runbook	Pre-tested operational steps for a known symptom/failure class.
RCA	Root-cause analysis explaining causal chain and prevention actions.
hypothesis	Possible explanation not yet proven by evidence.

Useful video / source route

[VIDEO: Google Cloud Tech - OpenTelemetry - Open video](#)

Watch: Optional visual - no verified bounded range

[VIDEO: Monte Carlo - Data Observability - Open video](#)

Watch: Optional visual - no verified bounded range

[OFFICIAL/CURRENT: oteI](#)

Use rule: Watch/read for the mental model or current interface, then return to this learner book and supplied practice. External media never replaces the independent task or validation evidence.

Follow with me once

Use a scenario: gold revenue becomes stale after deployment. Timeline facts show source arrived, raw load completed, silver validation failed, publish gate held the previous gold version. The runbook guides evidence collection and safe replay. The RCA then identifies a schema assumption introduced in the release and adds a contract test to prevent recurrence.

What you should see / why it worked

A timeline separating observed facts from hypotheses, containment action, recovery proof, customer/data impact, causal chain and prevention tasks with owners. The reason this works is that the control is explicit, measurable and separated from hidden assumptions. A trustworthy result is not merely an output; it includes the boundary that was checked and evidence that can be reviewed later.

Break it: a realistic failure

During an incident someone reruns the pipeline repeatedly with debug flags and deletes the failed batch to 'start clean.' This destroys evidence and may duplicate side effects. Safe response freezes relevant state, records commands and changes one variable at a time.

Reason through the incident

Work in this order: 1) preserve the failing evidence; 2) identify which contract or boundary failed; 3) bound blast radius before changing state; 4) choose the smallest reversible repair; 5) rerun an independent validation; 6) record what would prevent recurrence. If you skip evidence and jump straight to a fix, you may restore the dashboard while

losing the ability to explain why it failed.

Try it yourself

Write a runbook for 'curated row count drops >40%'. Include triage queries, stop/hold decision, replay boundary, escalation owner and proof of recovery.

Fresh changed retry

Change the symptom to high latency with correct output. Decide which containment and evidence steps differ and which remain the same.

Validation and recovery ladder

- PASS only when the required output and the named control both agree; a green command alone is insufficient.

- If the result is PARTIAL, first inspect inputs/configuration/permissions, then reproduce the smallest failing case.
- Never repair by deleting evidence or by granting broad privilege just to make an error disappear.
- After repair, change the fixture or scenario and prove the same principle still holds under a fresh case.

Revision in your own words

Core idea: Incidents, runbooks and root cause is about making production behavior explicit, reviewable and recoverable. The learner should be able to state the contract, show evidence, describe a failure boundary and perform a changed retry without copying the demonstration.

Teach it back

What is the difference between a runbook and an RCA? Why should facts and hypotheses be labeled separately during an incident?

Evidence to keep

Save: the genuine first attempt, any error/failure output, the corrected result, one validation proof, and a short explanation of why the control protects production. Do not overwrite the first failed evidence with the repaired run.

DE16-14

Terraform and IaC foundations

Watch first

[1. HashiCorp - Introduction to Terraform](#)

0:30-3:26 IaC/Terraform; 3:26-10:13 Day 0/1/2; 21:19-22:23 providers

[2. HashiCorp - Getting into Terraform, Part 1 \(2026\)](#)

Optional visual - no verified bounded range

Pause to connect the demonstration to the example below. Use the printed stop time when a bounded range is given; longer recordings remain available for reference.

Lesson 14 of 14 | Primary practice:

15_M16/practice/dataops_patterns/14_terraform_policy_check.py

Why this matters

Infrastructure as Code (IaC) expresses infrastructure intent in versioned files so changes can be reviewed, repeated and compared rather than reconstructed from memory and portal clicks. Terraform is one widely used IaC tool. For this course the stable baseline is Terraform 1.16.x; do not build the lesson around prerelease 1.17 behavior.

Picture it in your head

Think of a building blueprint plus change history. The blueprint does not physically construct the building by itself, but it gives reviewers an explicit desired design and a repeatable basis for changes.

Words before we use them

New word	Meaning before we use it
IaC	Managing infrastructure configuration through versioned declarative code.
provider	Plugin exposing resource types/data sources for a platform.
state	Terraform record used to map configuration to managed real resources.
plan	Proposed change set generated before apply.
drift	Difference between declared/recorded intent and real infrastructure.

Useful video / source route

[VIDEO: HashiCorp - Introduction to Terraform - Open video](#)

Watch: 0:30-3:26 IaC/Terraform; 3:26-10:13 Day 0/1/2; 21:19-22:23 providers

[VIDEO: HashiCorp - Getting into Terraform, Part 1 \(2026\) - Open video](#)

Watch: Optional visual - no verified bounded range

[OFFICIAL/CURRENT: terraform_install](#)

[OFFICIAL/CURRENT: terraform_cli](#)

[OFFICIAL/CURRENT: terraform_validate](#)

[OFFICIAL/CURRENT: terraform_test](#)

Use rule: Watch/read for the mental model or current interface, then return to this learner book and supplied practice. External media never replaces the independent task or validation evidence.

Follow with me once

Read the local sample configuration. It declares a Terraform version range $\geq 1.16, < 1.17$ for this controlled exercise and uses no real cloud credential. Walk the safe reasoning route: `terraform fmt`, `terraform init`, `terraform validate`, `terraform plan`, human/policy review, then apply only in an approved environment. The policy checker flags patterns such as wildcard admin intent in teaching examples.

```
terraform {
  required_version = ">= 1.16, < 1.17"
}

variable "environment" {
  type    = string
  default = "dev"
}

locals {
  labels = { environment = var.environment, course = "m16" }
```

What you should see / why it worked

You can explain what config, provider, state, plan and apply each do; you do not call a successful plan proof that the data pipeline itself is correct. The reason this works is that the control is explicit, measurable and separated from hidden assumptions. A trustworthy result is not merely an output; it includes the boundary that was checked and evidence that can be reviewed later.

Break it: a realistic failure

A team edits production resources manually after Terraform deploy. The next plan shows unexpected changes. Applying blindly can overwrite deliberate emergency fixes or propagate stale assumptions. First understand drift and ownership. State files can also contain sensitive values and require protected storage/access.

Reason through the incident

Work in this order: 1) preserve the failing evidence; 2) identify which contract or boundary failed; 3) bound blast radius before changing state; 4) choose the smallest reversible repair; 5) rerun an independent validation; 6) record what would prevent recurrence. If you skip evidence and jump straight to a fix, you may restore the dashboard while losing the ability to explain why it failed.

Try it yourself

Write a small IaC review checklist: formatting/validation, version constraints, plan review, least privilege, state protection, environment binding and post-deploy evidence.

Fresh changed retry

Change one harmless local variable in the sample configuration and predict the plan effect. Then imagine the same change affects a production storage account; list additional approvals and data-safety checks.

Validation and recovery ladder

- PASS only when the required output and the named control both agree; a green command alone is insufficient.
- If the result is PARTIAL, first inspect inputs/configuration/permissions, then reproduce the smallest failing case.
- Never repair by deleting evidence or by granting broad privilege just to make an error disappear.
- After repair, change the fixture or scenario and prove the same principle still holds under a fresh case.

Revision in your own words

Core idea: Terraform and IaC foundations is about making production behavior explicit, reviewable and recoverable. The learner should be able to state the contract, show evidence, describe a failure boundary and perform a changed retry without copying the demonstration.

Teach it back

Explain why terraform plan is valuable but does not prove application or data correctness. What extra evidence belongs after infrastructure deployment?

Evidence to keep

Save: the genuine first attempt, any error/failure output, the corrected result, one validation proof, and a short explanation of why the control protects production. Do not overwrite the first failed evidence with the repaired run.